

Slay the Roadmap / Slay the Code

GamiDOC di Progetto

D'Agostino (matr. 303226)

Di Giacomo (matr. 303377)

Corso di Esempi di Gamification e Sistemi Intelligenti (EGS)

A.A. 2025/2026 — 7 settembre 2026

Repository: https://github.com/Ago95Dev/slay_the_roadmap

*Nota sui nomi: progetto **Slay the Roadmap**, game Hub **Slay The Code** (game su console Hub).*

Indice

1	Context	3
1.1	Concept	3
1.2	Obiettivi didattici	3
1.3	Target users	3
1.4	Struttura della campagna	3
2	Game behavior	5
2.1	Core loop	5
2.2	Boss fight	5
2.3	Economie numeriche locali	6
3	Gamification	7
3.1	Meccaniche e dinamiche (Molina/Toda)	7
3.2	Lente Octalysis	7
4	Aesthetics e Dynamics	9
5	Architettura e scelte tecniche	10
5.1	Layer	10
5.2	Integrazione Hub (contratto)	11
5.3	Scelte tecniche	11
6	Stato implementazione (onesto)	12
6.1	Cosa gira davvero	12
6.2	Tagli dichiarati (future work)	12
7	Valutazione, KPI e Analytics	13
7.1	Valutazione (protocollo da eseguire)	13
7.2	KPI e Analytics	13

Capitolo 1

Context

1.1 Concept

Slay the Roadmap / Slay the Code è un dungeon crawler didattico offline-first realizzato in Flutter. La **roadmap** è il **dungeon**, i **topic** sono i **mostri/stanze**, le **carte-ricompensa** sono il **bottino**. Il giocatore studia un topic, supera un quiz e avanza lungo la roadmap fino al boss di capitolo. La campagna implementata è quella **Web – fundamenta** (file docs/assignment/campagna_web.md): zero sintassi, solo modelli mentali (perché/come/cosa-succede-se), quiz in italiano.

1.2 Obiettivi didattici

- Costruire i modelli mentali del Web prima della sintassi: come viaggiano le informazioni (rete), cosa sono e dove vivono (dati e stato), come si trasformano in pagine e si spediscono agli utenti (costruire).
- Dare feedback immediato e motivante: ogni quiz superato sblocca il nodo successivo e assegna una ricompensa; ogni capitolo si chiude con un boss.
- Rendere misurabile il progresso sia in locale (XP, livelli, vite, streak, save) sia sull'Hub (XP, badge, livelli remoti).

1.3 Target users

Studenti di programmazione con basi deboli o nulle di tecnologie Web, che devono orientarsi prima di scrivere codice. Non si assume conoscenza di HTTP, DNS, JSON, DOM o git: i testi dei topic partono dalle idee e i link di approfondimento (MDN, Cloudflare, web.dev, roadmap.sh) sono opzionali. Il gioco è pensato per sessioni brevi (pochi minuti per topic) e per essere rigiocabile da zero (New Run).

1.4 Struttura della campagna

Tre capitoli, ciascuno con un inquadramento (root) + 3 topic + 1 boss finale, per un totale di 12 nodi quiz. Tutti gli ID sono stabili in snake_case (Tab. 1.1, schema in Fig. 1).

Prerequisiti a catena: il capitolo 2 richiede la sconfitta del boss del cap. 1 (MITM), il capitolo 3 quella del boss del cap. 2 (Amnesiac); gli ID completi sono in Tab. 1.1. Ogni quiz (inclusi i root) ha ID `quiz_<topic>` e contiene 5 domande.

Capitolo	Nodi (root + 3 topic)	Boss
1 – La Rete	web_network, net_client_server, net_dns_url, net_http_https	man_in_the_middle
2 – Dati e Stato	web_data, data_represent, data_where, data_state	the_amnesiac
3 – Costruire sul Web	web_building, build_browser, build_framework, build_ship	spaghetti_colossus

Tabella 1.1: Mappa stabile della campagna Web.

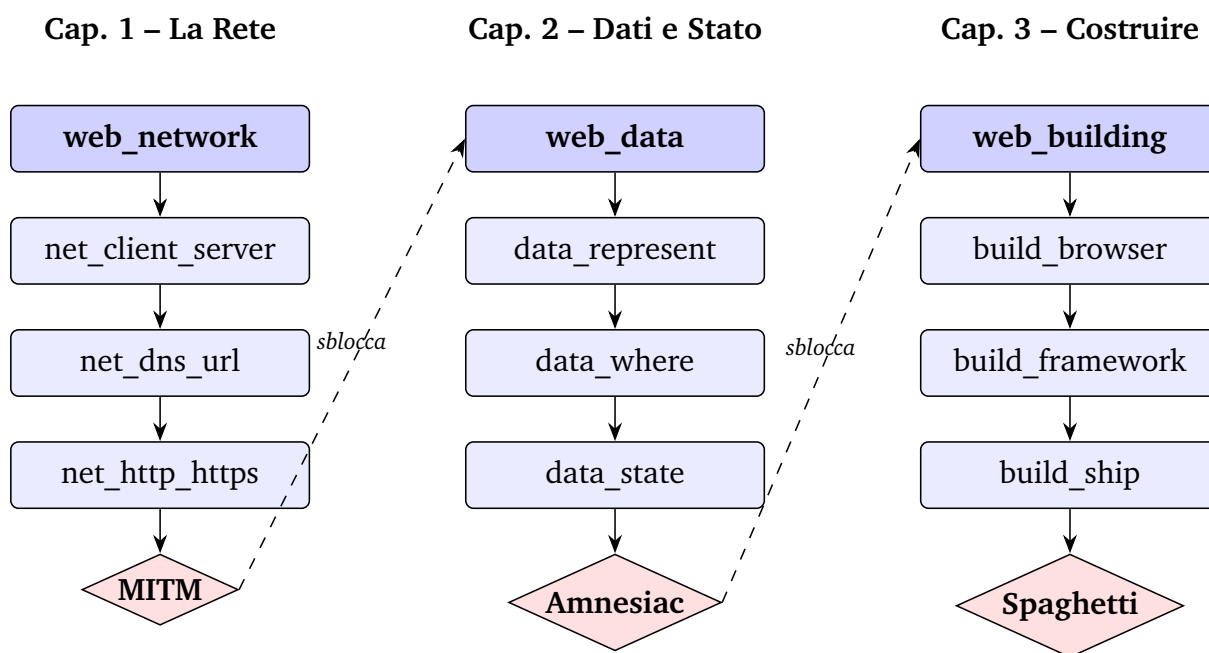


Figura 1: Albero della roadmap: 3 capitoli $\times$ (root + 3 topic) + boss di capitolo. La sconfitta del boss (MITM = man_in_the_middle, Amnesiac = the_amnesiac, Spaghetti = spaghetti_colossus) sblocca il capitolo successivo.

Capitolo 2

Game behavior

2.1 Core loop

Il loop (riassunto in Fig. 2) procede così:

1. **Studio:** pagina di dettaglio del topic (testo breve + link opzionali). Nessun timer.
2. **Quiz:** 5 domande a risposta multipla, soglia di superamento **80 % (4/5)**. I tentativi sono illimitati ma penalizzati (nessuna XP in caso di sconfitta/ripetizione fallita, perdita di vite nel boss).
3. **Ricompensa:** a quiz superato, scelta **1-di-3** carte (per tipo di ricompensa). Una sola ricompensa per topic; inventario visibile.
4. **Avanzamento:** lo sblocco del nodo successivo lungo la roadmap; a fine capitolo, **boss fight**.

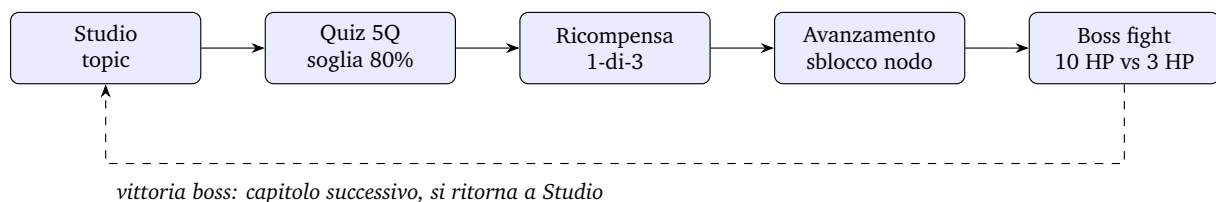


Figura 2: Core loop: Studio → Quiz → Ricompensa → Avanzamento → Boss fight, con ritorno al capitolo successivo.

2.2 Boss fight

Combattimento a turni con numeri reali (stato vero implementato):

- **Boss: 10 HP; giocatore: 3 HP; vite totali 3.**
- **Energia 3 per turno** (1 per carta giocata).
- A fine turno quiz obbligatorio: risposta giusta = danno al boss (quiz-danno), risposta errata = danno al giocatore.
- Vittoria: 1 ricompensa a scelta, sblocco del capitolo successivo, badge del boss. Sconfitta: retry a HP pieni, senza XP.

I tre boss (con lore di una riga dalla campagna Web):

- **Man-in-the-Middle** (cap. 1): “vive nei Wi-Fi aperti e legge ciò che non è cifrato”;
- **The Amnesiac** (cap. 2): “ha cancellato la memoria del web: ti costringe a ricordare tutto da solo, tra bigliettini, sessioni e copie stantie”;
- **Spaghetti Colossus** (cap. 3): “un ammasso di pagine copiate e mai versionate: ogni modifica spezza qualcos’altro”.

I quiz del boss sono adattivi: pescati dai 3 topic del capitolo.

2.3 Economie numeriche locali

- **XP/livelli:** soglie **L1 0 / L2 100 / L3 500**, identiche a quelle dell’Hub così che locale e remoto coincidano.
- **Streak:** bonus **+25 XP ogni 3 successi consecutivi**.
- **Vite:** 3. **Save locale:** SharedPreferences (Save v1: nodi completati, deck, XP, livello, boss sbloccati; Continue ripristina, New Run azzera con conferma).
- **Animazioni:** sobrie (transizioni essenziali, niente effetti extra).

Capitolo 3

Gamification

3.1 Meccaniche e dinamiche (Molina/Toda)

Gli elementi implementati, classificati secondo la tassonomia di Toda et al. [4]:

- **Performance:** punti XP, livelli (0/100/500), streak (+25/3), soglia quiz 80 %, feedback immediato con explanation come hint dopo errori ripetuti.
- **Ecological:** vite (3), energia boss (3/turno), economia chiusa senza valuta acquistabile.
- **Personal:** collezione carte (deck da ricompense 1-di-3), badge di topic e di boss, capitoli in sequenza con prerequisiti.
- **Fictional:** narrativa dei tre boss e del dungeon-roadmap.
- **Social:** tagliato (vedi Cap. 6); nessuna leaderboard locale, nessuna condivisione.

3.2 Lente Octalysis

Lettura secondo Chou [3]: il nucleo è **Sviluppo/Realizzazione (CD2)** nella soglia 80 % e nei livelli, **Creatività/Feedback (CD3)** nella scelta 1-di-3 e nel deck, **Possesso (CD4)** nelle carte e nei badge, **Imprevedibilità (CD7)** nella pesca boss dai topic del capitolo, **Perdita (CD8)** nel rischio boss (retry senza XP). Significato/Narrativa (CD1) è coperto dalla lore dei boss; Influenza sociale (CD5) e Scarsità (CD6) restano deboli per scelta di scope (visualizzazione dei pesi in Fig. 3).

DA COMPLETARE (Fase 1B, in corso). Le voci A–E del piano (narrativa CD1 estesa con intro/finale per capitolo, titoli + avatar, passive uniche dei boss, leaderboard Hub CD5, daily reward CD6) sono in corso di implementazione e NON sono ancora riflesse in questo capitolo. Questo paragrafo sarà aggiornato a chiusura della Fase 1B.

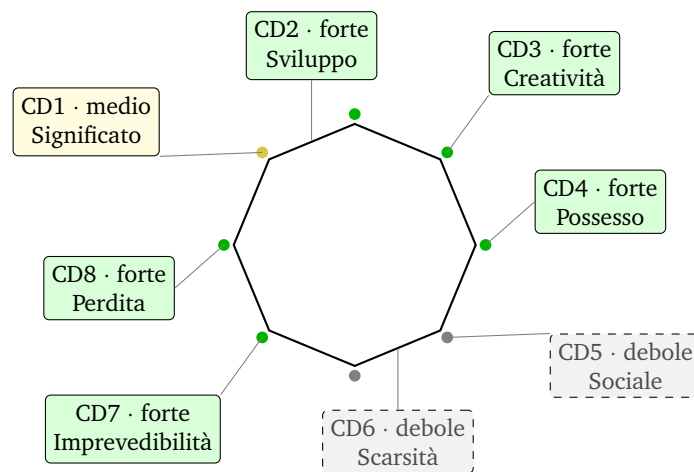


Figura 3: Ottagono Octalysis: pesi dei Core Drive nel progetto (dettagli nel testo).

Capitolo 4

Aesthetics e Dynamics

Estetica fantasy-dungeon sobria: la roadmap è la mappa del dungeon, i topic sono stanze da conquistare, i boss sono i guardiani di capitolo con nome e lore propri (Man-in-the-Middle, The Amnesiac, Spaghetti Colossus). Le rarità delle carte usano codice colore; le animazioni sono volutamente sobrie per non distrarre dallo studio.

Dynamics: la progressione è doppia ma semplice – **mastery** (XP, livelli, badge, sblocco nodi) e **potere** (deck più forte tramite ricompense 1-di-3), così che studiare meglio significhi combattere meglio. La soglia 80% mantiene il flow senza frustrare (tentativi illimitati ma penalizzati); il boss a 10 HP contro 3 HP del giocatore con quiz-danno a fine turno crea archi di tensione brevi e leggibili. Nessun dato di esperienza utente è ancora raccolto: si veda il Cap. 7.

Capitolo 5

Architettura e scelte tecniche

App Flutter offline-first. L'architettura è riassunta in Fig. 4; per i dettagli implementativi si rimanda al documento gemello `specifica.pdf`.

5.1 Layer

- **UI + Provider:** schermate (home, roadmap, topic, quiz, ricompensa, boss) con view-model Provider; un'unica fonte di verità locale per il progresso (PlayerProgress: XP, livello, vite, streak, deck, sblocchi).
- **Repository:** sei repository con responsabilità singola — roadmap (seed: 3 root + 9 sub, prerequisiti a catena tramite `bossId/requiredBossId`), testi dei topic (12), quiz (12 da 5 domande), boss (nomi, lore, quiz adattivi del capitolo), ricompense (scelta 1-di-3, limite 1/topic) e persistenza (save locale `Savev1` su `SharedPreferences`).
- **Engine:** logica di combattimento (HP 10 vs 3, energia 3, quiz-danno) e regole di sblocco, XP, streak e livelli isolate dai widget.
- **Hub client con fallback:** `EngineClient` reale e `FakeEngineClient` offline. Senza credenziali a runtime non parte alcuna chiamata di rete e l'app è identica (offline-first).

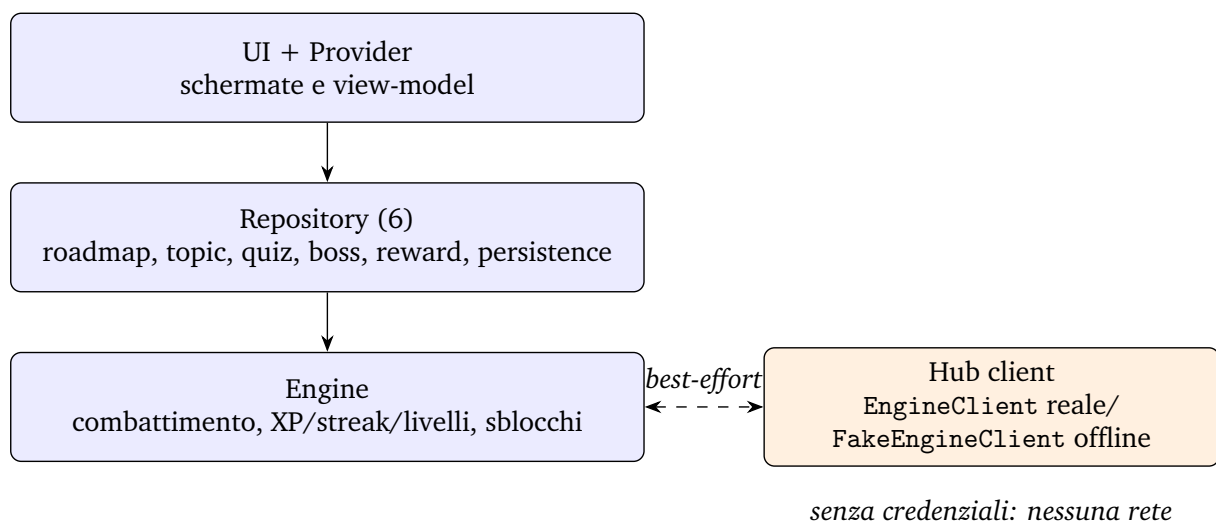


Figura 4: Schema a blocchi: UI/Provider e Repository appoggiano sull'Engine locale; l'Hub (reale o fake) è solo uno specchio best-effort di XP, badge e livelli, con fallback offline totale.

5.2 Integrazione Hub (contratto)

Game **Slay The Code** su console Hub (gameId = 6a9dbf66cc89679981fe7893, owner slay), API `gamification-api.createlab-univaq.it/api/v1` [1]. Risorse: action `quiz_completed`, `claim_reward`, `boss_defeated`; Point `xp`; Level `experience` su `xp` (L1 0, L2 100, L3 500); collezione badge `slay_badges`; rule `quiz_xp`, `topic_badge`, `boss_badge`. Eventi inviati `best-effort` dai view-model di `quiz` e `boss`, solo a `quiz` superato (soglia 80% verificata in app), con data sempre presente in `snake_case` (es. `xp_amount`, `badge`). Autenticazione a runtime via `--dart-define` (Bearer 24h), credenziali mai nel repo; `playerId` locale tipo `slay_<uuid>` salvato in `SharedPreferences`.

5.3 Scelte tecniche

Flutter + Provider per semplicità e testabilità; persistenza locale su `SharedPreferences` (nessun backend proprio); Hub come unico servizio remoto, usato solo come specchio di XP/badge/livelli con fallback totale. Ogni regola di `gamification` corrisponde a una action Hub, così che il GamiDOC resti fonte di verità dei parametri (soglie, danni, HP, costi).

Capitolo 6

Stato implementazione (onesto)

6.1 Cosa gira davvero

App Flutter offline-first: roadmap 3 capitoli $\times$ 3 topic + boss finali (12 nodi quiz da 5 domande, soglia 80 %), reward 1-di-3 per tipo, boss 10 HP / giocatore 3 HP / energia 3 con quiz-danno, vite 3, streak +25/3, XP/livelli 0/100/500, save locale SharedPreferences, animazioni sobrie. Hub: eventi quiz_completed/boss_defeated best-effort con XP/badge/livelli remoti e fallback offline completo.

6.2 Tagli dichiarati (future work)

Dichiarati nel piano (§2) e NON implementati: social e leaderboard locale, shop/coins, classi, login e signup (playerId locale fisso), roadmap extra, profilo/achievement avanzati, eventi testuali random, branching di campagna, difficoltà adattiva oltre la pesca boss, monthly quest e login reward, animazioni extra. Voci mockup M1 Sign-in e M2 Option2: rimosse o disattivate.

DA COMPLETARE (Fase 1B B–E, in corso). A chiusura delle voci B (titoli + avatar), C (passive uniche boss), D (leaderboard Hub) ed E (daily reward) aggiornare l'elenco “cosa gira” con i numeri reali implementati e i test corrispondenti. Nessun comportamento futuro va descritto al presente.

Capitolo 7

Valutazione, KPI e Analytics

7.1 Valutazione (protocollo da eseguire)

Think-aloud + questionari (SUS, IMI, nucleo GEQ) su 5–8 studenti, sessioni da circa 30 min. Sezioni *metodo* / *profilo partecipanti* / *risultati* / *riflessioni* da compilare dopo le sessioni.

DA COMPLETARE (User Evaluation da svolgere). Nessuna sessione è stata ancora svolta: nessun risultato utente viene riportato in questo documento. Compilare metodo, profili, risultati e riflessioni solo dopo le sessioni reali.

7.2 KPI e Analytics

KPI candidati: retention D1/D7, tasso di mastery al primo tentativo (soglia 80 %), tempo medio per boss, carte per vittoria, $SUS \geq 70$. Eventi tracciabili via Hub (data nelle executions): `quiz_completed`, `claim_reward`, `boss_defeated`. Le soglie di successo saranno fissate dopo la prima tornata di dati reali.

Bibliografia

- [1] Gamification Hub Univaq, console <https://gamification-webapp.createlab-univaq.it> e API <https://gamification-api.createlab-univaq.it/api/v1> (contratto eventi in docs/assignment/hub_setup.md).
- [2] A. Molina et al., materiali GamiDOC e user research (think-aloud + questionari), corso EGS.
- [3] Y.-K. Chou, *Actionable Gamification*, Octalysis Media, 2015.
- [4] A. M. Toda et al., “A taxonomy of game elements for gamified applications”, *SIGCSE*, 2019.